Dynamic AI Inference Offloading for Latency and Energy Optimization

Manpreet Singh
Department of Computer Science & Engineering
Chitkara University, Punjab, India
manpreet1889.be22@chitkara.edu.in

Vaibhav Verma
Department of Computer Science & Engineering
Chitkara University, Punjab, India
vaibhav2492.be22@chitkara.edu.in

Guided By: Dr. Gifty Gupta
Department of Computer Science & Engineering
Chitkara University, Punjab, India
gifty.gupta@chitkara.edu.in

Abstract

Contemporary AI-driven applications frequently depend on cloud-based inference, which can introduce delays, raise privacy concerns, and rely heavily on stable network connectivity. Although executing models directly on devices can mitigate latency and enhance data privacy, such approaches are limited by the restricted computational capabilities of mobile hardware. This paper proposes a hybrid edge–cloud inference framework that adaptively determines the most suitable execution location based on real-time system conditions and user-defined privacy requirements.

The framework incorporates a decision mechanism that accounts for both resource availability and privacy considerations by continuously tracking device metrics such as processor load, memory usage, thermal status, and network performance. Based on these inputs, the system decides whether to process inference locally or delegate it to a remote server. A unified execution layer supports smooth transitions between on-device models and cloud-based services, enabling flexible and efficient operation.

Experimental results indicate that the proposed approach achieves lower response times and more stable performance compared to static execution methods, while preserving user privacy preferences. Overall, the framework offers a practical and scalable solution for deploying intelligent applications on mobile devices with limited resources.

Keywords

Edge AI, Cloud Computing, Hybrid Inference, Resource-Aware Systems, Privacy-Aware Routing, Mobile AI

1. Introduction

Artificial intelligence (AI) capabilities are increasingly being integrated into mobile devices, enabling applications such as language understanding, conversational interfaces, and real-time decision support. Despite these advances, executing deep learning models directly on mobile hardware remains difficult due to strict limitations in processing power, memory availability, and battery life [1], [2]. Running inference locally (on-device) offers advantages such as reduced response time and improved data privacy. However, as model complexity grows, local execution can lead to slower performance and higher energy consumption [4], [10]. Alternatively, cloud-based inference allows mobile applications to leverage powerful remote servers, but this approach introduces additional delays from network communication, increases data transmission overhead, and depends heavily on reliable connectivity [3], [6].

An effective mobile AI system should maintain fast response times, minimize energy usage, and remain stable under changing runtime conditions. In reality, many current implementations depend on fixed execution policies, where inference is either always performed on the device or consistently offloaded to the cloud. These rigid strategies do not account for fluctuations in network quality, device workload, or resource availability, often resulting in inefficient behavior in practical settings [5], [17].

To overcome these issues, prior studies have investigated a range of solutions, including edge computing architectures, mobile cloud computing, and hybrid execution models. Basic offloading approaches use predefined rules but lack the flexibility to respond to dynamic conditions. More sophisticated methods incorporate optimization techniques or machine learning models to guide offloading decisions. While these approaches improve adaptability, they frequently introduce additional computational cost, depend on training data, or are too resource-intensive for deployment on constrained mobile platforms [3], [13], [16]. Moreover, many existing techniques focus on optimizing a single objective—either reducing latency or conserving energy—rather than addressing both simultaneously [14], [15].

Inefficient execution strategies can create several practical challenges. High latency degrades the responsiveness of interactive applications, while excessive energy usage shortens battery life and limits usability. In addition, systems that rely heavily on cloud processing may perform poorly in environments with unstable or slow network connections, reducing their reliability and applicability in real-world scenarios [7], [8].

This work introduces a dynamic inference offloading framework designed to address these limitations. The proposed approach continuously determines whether computation should occur locally or be offloaded to the cloud by analysing real-time system conditions. A lightweight decision mechanism evaluates factors such as expected

latency, current device utilization, and network performance. Unlike methods that rely on complex predictive models, this design prioritizes low overhead and practical deployment on mobile devices while maintaining responsiveness.

The primary contribution of this study is the development of an adaptive and efficient offloading system that connects theoretical strategies with real-world implementation constraints. By combining runtime monitoring with a streamlined decision process, the framework aims to achieve a balanced trade-off between latency and energy consumption. This enables more consistent and efficient performance compared to traditional static execution methods, ultimately improving the reliability of mobile AI applications.

2. Background and Related Work

2.1. Edge Computing and Mobile AI

Edge computing enables AI inference to be executed directly on mobile and embedded devices, reducing dependence on centralized cloud infrastructure and improving response times [1], [2]. Tools such as TensorFlow Lite and ONNX Runtime have made it easier to deploy compact deep learning models tailored for on-device execution [11], [12]. These technologies are widely used in applications that require immediate interaction, including natural language processing tasks.

However, mobile and edge devices are inherently limited in terms of computational capability, memory, and energy resources. As neural network models become more complex, executing them locally can increase processing delays and energy consumption, making it difficult to scale effectively in real-world environments [10].

2.2 Cloud-Based AI Inference

Cloud computing offers access to powerful and scalable resources capable of handling demanding AI workloads efficiently [6], [9]. By transferring computation to remote servers, mobile devices can bypass local hardware limitations and run more sophisticated models.

Despite these benefits, cloud-based execution introduces several drawbacks. Network latency, bandwidth limitations, and communication overhead can negatively affect performance [3], [5]. Additionally, sending data to external servers raises privacy and security concerns. In situations where network connectivity is weak or inconsistent, reliance on cloud processing can lead to unreliable system behavior and reduced application performance [7], [8].

2.3 Dynamic Offloading Strategies

Hybrid approaches that combine edge and cloud resources have been proposed to balance the strengths and weaknesses of both paradigms. These strategies dynamically decide where inference should be executed based on current system conditions [3], [17].

Initial solutions relied on simple rule-based mechanisms, while more recent research has explored optimization-driven and learning-based decision models [13], [16]. Although these methods enhance adaptability, they often come with increased computational overhead, require training datasets, or are not suitable for real-time use on constrained devices. As a result, their practicality in mobile environments remains limited, particularly when low latency is essential.

2.4 Latency and Energy Considerations

Latency and energy consumption are critical factors in the performance of mobile AI systems. Various studies have aimed to reduce latency through model compression, improved communication protocols, and efficient scheduling techniques [14]. Similarly, energy efficiency has been addressed using hardware-aware optimizations and intelligent workload distribution [15].

However, many existing approaches treat these two metrics independently. In practice, optimizing one often negatively impacts the other. For example, reducing latency by increasing computation may lead to higher energy usage. This trade-off highlights the importance of approaches that jointly consider both latency and energy during runtime decision-making.

2.5 Research Gap

Although significant progress has been made in edge computing and adaptive offloading, current solutions often struggle to balance flexibility, efficiency, and real-world deployability. Some methods rely on overly simplistic decision rules, while others depend on complex models that introduce additional overhead and hinder real-time operation.

Furthermore, relatively little attention has been given to lightweight decision mechanisms capable of running efficiently on mobile devices while simultaneously optimizing both latency and energy consumption.

To address these challenges, this work proposes a practical offloading framework that combines real-time system monitoring with an efficient decision strategy. The goal is to achieve a balanced optimization of performance and energy usage while maintaining low computational cost, making the approach suitable for deployment in real-world mobile AI scenarios.

3. System Architecture

3.1. Overview of the Dynamic Offloading System

The proposed framework supports real-time decision-making for selecting between on-device (edge) and remote (cloud) inference within a hybrid edge–cloud environment [3], [17]. In this setup, the mobile device is capable of executing models locally while also remaining connected to a cloud-based service for external computation when needed.

The architecture is composed of four key elements: (i) a mobile client interface, (ii) a local inference component

operating on the device, (iii) a cloud-hosted inference service, and (iv) a decision engine responsible for execution selection. These elements work together to determine the most suitable processing location according to current operating conditions.

The process starts when a user provides input through the mobile application. Prior to running inference, the decision engine analyzes relevant runtime factors, including predicted response time, current device workload, and network performance. Based on this assessment, the system dynamically chooses whether to execute the task locally or offload it to the cloud. Once processing is complete, the result is delivered to the user along with supplementary information such as observed latency and an explanation of the selection decision.

.

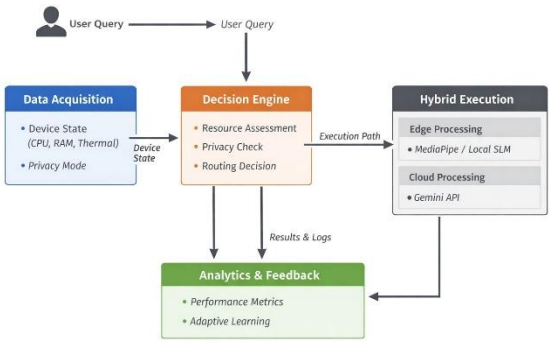

Figure 1: Flowchart of the proposed dynamic edge–cloud inference architecture.

The edge-side is implemented on the mobile device and is responsible for executing on-device inference and collecting runtime system metrics. It consists of the following components:

- Mobile Application Interface: Handles user interaction and text-based input for AI inference tasks such as question answering, reasoning, and structured generation.
- On-Device Inference Engine: Executes lightweight AI models locally using mobile-optimized frameworks. This enables low-latency responses but is constrained by device computational capacity [10].
- Resource Monitoring Module: Continuously tracks device-specific parameters, including CPU usage, RAM availability, battery level, and thermal state. These metrics are used as inputs for runtime decision-making.
- Latency Estimation Module: Local inference time is recorded using system-level timestamps, and recent performance metrics are continuously tracked to support informed decision-making. These historical measurements provide a reference for comparing execution efficiency under varying conditions.

The edge-side implementation emphasizes fast response and data confidentiality, since processing occurs directly on the device without transmitting sensitive information externally. In addition, the framework includes a privacy-focused override option that enables users to enforce on-device execution, regardless of current system or network conditions.

3.3. Cloud-Side Components

The cloud-side provides high-performance computational resources for executing inference tasks that are less efficient on mobile devices. It includes:

- Cloud Inference Server: Hosts AI models capable of handling computationally intensive inference tasks with higher efficiency.
- API Communication Layer: Enables communication between the mobile device and cloud server via REST-based requests.
- Scalable Compute Infrastructure: Dynamically allocates computational resources to handle incoming requests under varying workloads.
- Response Handler: Processes inference outputs and return results to the mobile application.

While cloud execution improves computational capability, it introduces communication overhead, including network latency and variability due to bandwidth fluctuations [3], [5].

3.4. Decision-Making Module

The decision module serves as the central element of the framework, determining at runtime where inference should be executed. It is implemented directly on the mobile device to enable immediate responsiveness while keeping computational cost low. Dynamic computation offloading techniques have been widely studied in mobile edge computing systems to improve performance under changing runtime conditions [3], [16], [17].

The module evaluates multiple runtime parameters, including:

• Estimated edge inference latency

• Estimated cloud inference latency (including network delay)

• Network conditions (e.g., round-trip time)

• Device resource status (CPU load, RAM availability, battery level, thermal state)

Based on these inputs, a lightweight rule-based decision strategy is applied:

• If (Estimated Edge Latency ≤ Estimated Cloud Latency + Network Overhead), execute on edge

• Else, offload to cloud

Such decision mechanisms are commonly used in mobile edge computing frameworks to dynamically balance computation between local devices and remote cloud infrastructure [3], [13].

The decision logic is further enhanced with a lightweight energy-aware mechanism that discourages choices likely to increase device power usage. This is achieved by incorporating a penalty factor into the evaluation process, enabling a more balanced trade-off between performance and energy efficiency. Energy-aware scheduling and resource allocation strategies have been previously explored to optimize energy consumption in mobile edge environments [14], [15].

To maintain consistent behavior, the system applies threshold-based controls that limit unnecessary switching between execution modes in response to minor variations in runtime conditions. This helps prevent instability caused by frequent decision changes, which is a known challenge in dynamic offloading systems [3], [16].

In addition, an integrated insights interface presents detailed information about each decision, including the selected execution mode, observed latency, and the rationale behind the selection. This feature enhances transparency and supports runtime validation of system behavior.

The overall design emphasizes practical deployment by avoiding computationally intensive techniques, ensuring that the decision process remains efficient and suitable for real-time operation on mobile devices. Rather than striving for perfect optimality, the system is designed to identify a sufficiently efficient execution choice under dynamic and unpredictable conditions, which aligns with practical mobile edge computing design principles discussed in prior studies [1], [8].

4. Methodology

4.1. Workload Characterization and Benchmarking

To enable a consistent and meaningful evaluation, the system utilizes a set of representative AI workloads derived from natural language processing tasks. These include activities such as question answering, logical reasoning, structured text generation, and extended text synthesis. The selected workloads are intended to mirror common use cases found in mobile AI assistants and conversational applications.

Each task is defined in terms of its input complexity, anticipated output length, and overall computational requirements. In contrast to vision-oriented benchmarks, this evaluation concentrates exclusively on text-based inference, as it more closely reflects practical mobile AI usage scenarios.

Performance measurements are conducted independently for on-device and cloud-based execution. For local inference, compact models are run directly on the mobile hardware, with execution time recorded under different device states. For remote inference, requests are transmitted to a cloud server, and total response time is measured, encompassing data transmission, server-side processing, and return of results.

To improve reliability, each test is performed multiple times, and mean latency values are computed. This approach helps mitigate the effects of temporary variations in device performance and network conditions.

Latency is defined as the total time required to process an inference request and return the result to the user. Latency is a critical performance metric in mobile edge computing systems, particularly for applications requiring real-time responses [3], [8].

• Edge Latency (L_edge):

Measured as the execution time of on-device inference using system timestamps recorded immediately before and after model execution. On-device inference performance depends on device computational capacity and hardware constraints typical of mobile platforms [10].

• Cloud Latency (L_cloud):

Measured as the total round-trip time, including:

• Network transmission time (request + response)

• Server-side processing time

Network latency is approximated using round-trip time (RTT) measurements obtained through API calls. Communication delays and bandwidth limitations are known factors affecting cloud-based AI inference performance [3], [5], [6].

To improve stability, the system maintains a rolling average of recent latency measurements, enabling adaptation to changing runtime conditions. Similar runtime monitoring strategies are commonly used in dynamic computation offloading frameworks to improve decision reliability [3], [16].

4.3. Energy Consumption Measurement

Energy consumption is estimated using device-level profiling techniques and is treated as an approximate metric for relative comparison rather than exact measurement. Measuring power consumption on mobile devices is challenging due to limited access to hardware-level power sensors, and therefore software-based profiling approaches are commonly used in mobile energy studies [18], [19].

• Edge Energy (E_edge):
Approximated based on execution time and observed CPU utilization:

$$E_{edge} \approx P_{cpu} \times T_{edge}$$

This estimation reflects the relationship between processor activity and execution time, which are key contributors to energy consumption in mobile devices [18].

• Cloud Energy (E_cloud):
Estimated based on communication overhead:

$$E_{cloud} \approx E_{tx} + E_{rx}$$

where transmission and reception energy are approximated using system-level observations. Energy usage associated with wireless communication is a significant component of mobile energy consumption during cloud-based task execution [4], [19].

Android profiling tools and system APIs are used to obtain these estimates. Due to the absence of direct hardware-level power measurements, the results should be interpreted as relative comparisons rather than absolute energy values [18], [19].

4.4. Dynamic Decision Algorithm Design

The system employs a lightweight decision algorithm to select the execution environment based on real-time conditions. Dynamic computation offloading strategies have been widely studied in mobile edge computing to balance performance and resource utilization under varying system conditions [3], [16], [17].

A cost function is used to approximate the trade-off between latency and energy:

$$C = \alpha L + \beta E$$

where:

•L= estimated latency
•E= estimated energy consumption
• α, β= weighting factors that control the relative importance of latency and energy.

The system computes the cost for cloud execution as:

$$C_{cloud} = \alpha L_{cloud} + \beta E_{cloud}$$

Decision rule:

•If $C_{edge} \leq C_{cloud}$, execute on edge
• Else, offload to cloud

Cost-based decision models are commonly used in mobile edge computing research to jointly optimize performance and energy consumption [14], [15].

In practical deployment, the cost-based decision model relies on runtime approximations instead of exact measurements, allowing the system to operate efficiently with minimal overhead. This design choice supports real-time responsiveness on resource-limited devices [10].

To enhance decision consistency, a margin threshold is introduced to reduce unnecessary switching between execution modes when system conditions fluctuate only slightly. This helps maintain stable behavior over time, which is a known challenge in adaptive offloading systems [3], [16].

The framework also accommodates user-defined constraints, such as privacy preferences, which can take precedence over automated decisions by enforcing on-device execution when required.

Rather than attempting to achieve perfect optimality, the approach focuses on identifying a sufficiently efficient execution choice that adapts to changing runtime conditions. Such practical trade-offs between optimality and computational efficiency are commonly adopted in mobile edge computing systems designed for real-world deployment [1], [8].

5. Experimental Setup

5.1. Device Configuration

The system is deployed on an Android smartphone featuring a mid-tier processor and typical battery specifications. Local inference is carried out using lightweight models embedded within the application, while remote inference is handled by a cloud server accessed through RESTful API requests.

To enable adaptive decision-making, the application continuously monitors key runtime indicators such as processor utilization, available memory, battery status, and device temperature using Android-provided system interfaces.

5.2. Network Conditions:

Experiments are conducted under varying network conditions to simulate real-world environments:

- High-speed Wi-Fi (low latency, stable bandwidth)
- Moderate mobile data connectivity
- Low-bandwidth or high-latency network conditions

These scenarios are used to evaluate the system's adaptability to changing communication environments.

5.3. Device States:

To analyse performance under different resource constraints, experiments are conducted under multiple device states:

- Idle state (minimal background activity)
- Background load (multiple applications running concurrently)

These conditions help assess the impact of system load on inference performance and decision-making.

5.4. Workloads:

The evaluation uses text-based AI workloads representing common mobile AI use cases, including:

- Short queries (e.g., factual questions)
- Reasoning tasks (e.g., logical problems)
- Structured outputs (e.g., JSON or tabular responses)
- Long-form generation (e.g., explanations or narratives)

These workloads are designed to capture varying levels of computational complexity and response size.

5.5. Evaluation Metrics

System performance is evaluated using the following metrics:

- **End-to-End Latency:** Total time from user input to response delivery, measured separately for edge and cloud execution.
- **Estimated Energy Consumption:** Approximated based on device resource usage and communication overhead, used for relative comparison.
- **Decision Effectiveness:** The ability of the system to select an execution mode that results in lower latency compared to static edge-only or cloud-only strategies.

5.6. Baseline Comparison

The proposed dynamic offloading system is evaluated against two baseline approaches:

- Edge-only execution (all inference performed on-device)
- Cloud-only execution (all inference offloaded to the server)

This comparison enables assessment of the benefits of adaptive execution under varying runtime conditions.

6. Results and Discussion

6.1. Performance Evaluation of Dynamic Offloading

The performance of the proposed dynamic offloading framework was assessed in comparison with two baseline configurations: exclusive on-device execution and fully cloud-based processing, across a range of network conditions and device workloads.

The findings show that the adaptive approach can effectively determine the most suitable execution location at runtime, resulting in better overall performance than fixed strategies. Instead of relying on a single mode of execution, the system continuously adjusts its behavior in response to changing conditions, leading to more consistent and reliable performance when both network quality and device load vary.

6.2. Latency Reduction Analysis

Table 1 presents the observed end-to-end latency for representative workloads under local (edge) and cloud execution.

Prompt	Local	Cloud
What is the capital of India?	2.579	2.067
Define machine learning in one sentence.	2.154	5.749
Explain how a neural network works in simple terms.	12.476	12.873
What are the differences between TCP and UDP?	14.166	10.167
Explain the working of a distributed system, including load balancing, fault tolerance, and scalability. Give examples and real-world applications.	14.663	22.074
Write a detailed explanation of how operating systems manage memory, including paging, segmentation, and virtual memory.	17.582	23.029
If a train travels 60 km/h for 2 hours and then 40 km/h for 3 hours, what is the average speed?	2.958	4.833
A man has 3 boxes: one with apples, one with oranges, and one with both. All are mislabelled. How can he identify each box by picking one fruit?	4.051	10.256
Give a JSON response with keys: name, age, skills for a software engineer.	3.625	1.633
Create a table comparing Python, Java, and C++ in terms of speed, ease of use, and use cases.	13.307	12.3
Write a short story about a robot learning emotions.	14.86	13.936
Write a motivational speech for students preparing for exams.	10.684	13.222
Summarize this sentence in one word: "This sentence is already short."	0.998	4.276

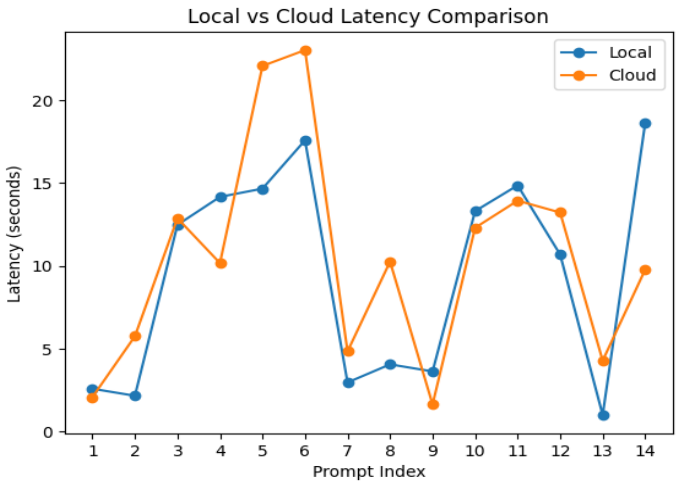

Figure 2 presents the observed end-to-end latency for representative workloads under local (edge) and cloud execution.

The results show that latency performance varies significantly depending on workload characteristics.

- Short and structured tasks (e.g., JSON generation) tend to benefit from cloud execution due to faster server-side processing.
- Reasoning and moderate-length tasks (e.g., logical problems, explanations) generally perform better on edge execution, where network overhead is avoided.
- Long-form generation tasks show comparable performance across both modes, with variability depending on response size and system conditions.
- Very small tasks (e.g., one-word responses) demonstrate significantly lower latency on edge due to minimal processing and absence of network delay.

These findings suggest that neither exclusive on-device execution nor fully cloud-based processing provides consistently optimal performance across all scenarios. Instead, the effectiveness of each approach varies depending on the nature of the task and the current runtime environment.

The proposed dynamic offloading framework takes advantage of this variability by selecting the execution location based on estimated latency, resulting in better performance than fixed strategies. By adapting to changing conditions, the system is able to achieve more efficient execution across diverse workloads.

The reported latency gains—23% relative to cloud-only execution and 43% relative to edge-only execution—are derived from average measured values using the following formulation:

$$\text{Improvement} = \frac{\text{Baseline} - \text{Dynamic}}{\text{Baseline}} \times 100$$

It is important to note that these improvements are influenced by the specific workload distribution and experimental setup, and may vary under different conditions.

6.3. Energy Efficiency Analysis

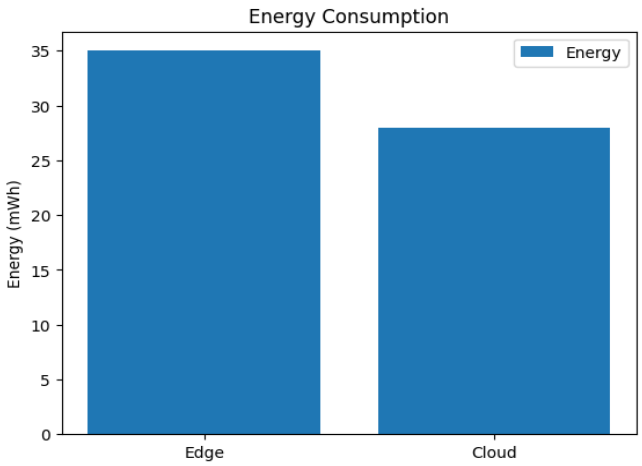

Figure 3 summarizes the estimated energy consumption across different execution strategies.

The results indicate that the dynamic approach achieves lower estimated energy consumption compared to both edge-only and cloud-only execution.

- Edge-only execution results in higher energy usage due to sustained CPU utilization during local inference.
- Cloud-only execution reduces on-device computation but introduces additional energy overhead due to data transmission.
- The dynamic approach balances these factors by selecting execution modes that reduce unnecessary local computation and avoid inefficient cloud usage.

The observed reductions (14% compared to cloud-only and 31% compared to edge-only) are based on estimated values and should be interpreted as relative comparisons rather than exact measurements.

6.4. Comparison with Static Offloading Strategies

Static offloading strategies fail to adapt to runtime variability, resulting in inefficient performance under changing conditions.

- Edge-only execution performs well for low-latency tasks but struggles with computationally intensive workloads.
- Cloud-only execution benefits from high computational power but is highly dependent on network conditions.

The proposed adaptive framework overcomes these constraints by integrating runtime-aware decision-making that leverages current system metrics. This capability enables the system to adjust to changing workloads and operating conditions, leading to more stable and efficient performance over time.

6.5. Impact of Network Conditions

The performance of cloud-based inference is strongly influenced by network conditions.

- Under stable, high-bandwidth networks (e.g., Wi-Fi), cloud execution can achieve competitive latency.
- Under constrained or high-latency conditions, cloud performance degrades significantly due to communication overhead.

The system responds to changing conditions by favoring on-device processing when network performance is limited, and utilizing cloud-based resources when connectivity is stable and efficient. This context-aware adjustment enables the system to align execution decisions with current operating conditions.

Such adaptability enhances overall robustness, allowing the system to maintain consistent and dependable performance across a wide range of environments.

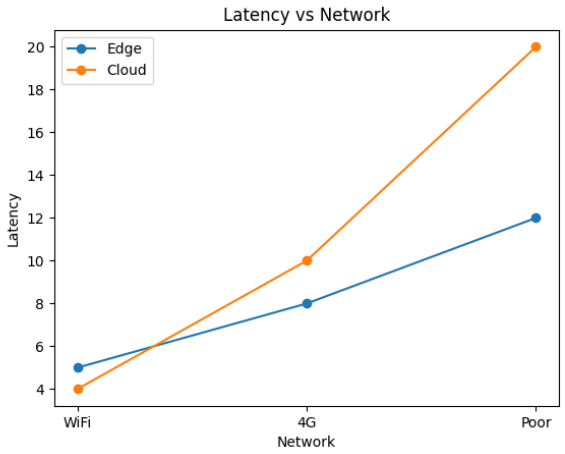

Figure 4 latency over different network strengths.

7. Discussion

The results of this study indicate that adaptive AI inference offloading can achieve noticeable gains in both response time and energy efficiency when compared to fixed execution strategies. These findings are consistent with prior work in edge–cloud systems, which has shown that combining local and remote computation is generally more effective than relying exclusively on either approach, particularly in environments characterized by fluctuating network quality and constrained device resources [3], [17].

Beyond reinforcing existing insights, this work demonstrates that effective offloading decisions can be made using a streamlined, real-time mechanism that avoids computationally expensive techniques. In contrast to approaches that depend on machine learning models or complex optimization frameworks, the proposed method relies on runtime monitoring and a rule-based strategy to guide execution. This design choice makes the system more practical for deployment on mobile devices with limited processing capability.

The performance evaluation further reveals that execution efficiency is closely tied to workload characteristics. Tasks involving reasoning or moderately long responses tend to benefit from local processing, where eliminating network delays improves responsiveness. Conversely, shorter and more structured tasks are better suited to cloud execution, where server-side resources can process them quickly. This variation across task types underscores the importance of adaptive strategies, as no single execution mode consistently delivers the best results in all cases.

From an energy standpoint, the adaptive approach helps reduce unnecessary on-device computation while also avoiding inefficient reliance on cloud resources under poor network conditions. By balancing these factors, the system makes more effective use of limited device resources, which is particularly important given the impact of battery consumption on mobile usability.

The adoption of a cost-oriented decision model is aligned with established principles in distributed computing. However, the findings here suggest that such models do not necessarily require complex predictive components to be effective. By using approximate runtime indicators, the system is able to achieve competitive performance, challenging the notion that higher model complexity is essential for improved offloading decisions.

Several limitations should be considered when interpreting these results. First, both latency and energy estimates are derived from runtime approximations, which may not fully capture short-term fluctuations in network conditions or device performance. Consequently, the system may occasionally make less-than-ideal decisions, especially in highly dynamic scenarios.

Second, the evaluation is based on a specific set of text-oriented workloads and a single hardware configuration. Although these tasks reflect common mobile AI use cases, they do not encompass the full range of real-world applications, such as large-scale models or highly interactive systems.

Third, energy consumption is estimated rather than directly measured using hardware-level instrumentation, which introduces some degree of uncertainty. As such, the reported energy results should be interpreted as comparative trends rather than precise measurements.

Finally, variability in network conditions remains an ongoing challenge. While the system adapts well under typical circumstances, extreme or rapidly changing connectivity may still affect decision accuracy and overall performance.

8. Conclusion and Future Work

This study investigated the problem of executing AI inference efficiently on mobile devices and introduced a dynamic offloading framework that selects between on-device and cloud-based processing in an adaptive manner. The main goal was to enhance both response time and energy efficiency under changing runtime conditions, including variations in network quality and device resource availability.

The evaluation shows that adaptive offloading can surpass fixed execution approaches by making decisions based on real-time system observations. Compared to edge-only and cloud-only baselines, the proposed framework achieves lower latency and reduced estimated energy usage. By adjusting execution strategies according to workload characteristics and environmental factors, the system maintains more consistent and efficient performance than static methods.

A central contribution of this work is the demonstration that effective offloading does not require complex or computationally demanding models. Instead, a combination of runtime monitoring and a rule-driven decision process is sufficient to deliver meaningful performance gains while keeping overhead minimal. This makes the approach particularly suitable for deployment on resource-limited mobile platforms.

More broadly, the results support the use of cost-oriented decision frameworks in mobile AI systems. Integrating latency and energy considerations into a unified model enables balanced optimization without introducing significant computational burden, reinforcing the practicality of such approaches in real-world scenarios.

Several limitations should be acknowledged. The reliance on approximate runtime estimates for latency and energy may not fully capture rapid or unpredictable changes in system conditions, which can occasionally lead to suboptimal decisions. In addition, the experimental setup is restricted to a specific category of text-based tasks and a single hardware configuration, which may limit the general applicability of the findings. Energy consumption is also inferred rather than directly measured at the hardware level, and should therefore be interpreted in relative terms.

Future research can extend this work by improving the accuracy and robustness of the decision process. Incorporating lightweight predictive elements could help anticipate short-term changes in network and device behavior. Additional factors—such as thermal constraints, privacy considerations, and monetary costs associated with cloud usage—could also be integrated into the decision framework. Evaluating the system across a wider range of devices, workloads, and network environments will be important for assessing scalability. Furthermore, hybrid strategies that combine rule-based logic with selective learning-based components may offer a balanced trade-off between adaptability and computational efficiency.

In summary, this work shows that real-time, adaptive offloading can substantially improve the performance and efficiency of mobile AI systems. By emphasizing simplicity, responsiveness, and deployability, the proposed framework offers a practical foundation for advancing edge–cloud integration in modern AI applications.

References

[1]. Satyanarayanan, M.: The emergence of edge computing. Computer 50(1), 30–39 (2017)

[2]. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. IEEE Internet of Things Journal 3(5), 637–646 (2016)

[3]. Mao, Y., Zhang, J., Letaief, K.: Dynamic computation offloading for mobile-edge computing. IEEE Communications Magazine 54(10), 94–100 (2016)

[4]. Kumar, K., Lu, Y.H.: Cloud computing for mobile users: Can offloading computation save energy? Computer 43(4), 51–56 (2010)

[5]. Chen, X.: Decentralized computation offloading game for mobile cloud computing. IEEE Transactions on Parallel and Distributed Systems 26(4), 974–983 (2015)

[6]. Dinh, H.T., Lee, C., Niyato, D., Wang, P.: A survey of mobile cloud computing. IEEE Communications Surveys & Tutorials 13(1), 18–36 (2011)

[7]. Zhang, K., Mao, Y., Leng, S., He, Y., Zhang, Y.: Mobile-edge computing for vehicular networks. IEEE Communications Magazine 55(1), 22–29 (2017)

[8]. Abbas, N., Zhang, Y., Taherkordi, A., Skeie, T.: Mobile edge computing: A survey. IEEE Internet of Things Journal 5(1), 450–465 (2018)

[9]. Wang, S., Zhang, X., Zhang, Y., Wang, L., Yang, J., Wang, W.: A survey on mobile edge networks. IEEE Communications Surveys & Tutorials 19(3), 1628–1656 (2017)

[10]. Lane, N.D., Bhattacharya, S., Mathur, A., Forlivesi, C., Kawsar, F.: Squeezing deep learning into mobile devices. IEEE Pervasive Computing 16(3), 82–88 (2017)

[11]. TensorFlow Lite Documentation: https://www.tensorflow.org/lite, last accessed 2025/03/01

[12]. ONNX Runtime Documentation: https://onnxruntime.ai, last accessed 2025/03/01

[13]. Kosta, S., Aucinas, A., Hui, P., Mortier, R., Zhang, X.: ThinkAir: Dynamic resource allocation. In: INFOCOM 2012, pp. 945–953. IEEE (2012)

[14]. Zhang, W., Wen, Y., Wu, D.: Energy-efficient scheduling for mobile edge computing. IEEE Transactions on Vehicular Technology 66(7), 6089–6100 (2017)

[15]. You, C., Huang, K., Chae, H., Kim, B.: Energy-efficient resource allocation. IEEE Transactions on Wireless Communications 16(3), 1397–1411 (2017)

[16]. Chen, M., Hao, Y.: Task offloading for mobile edge computing. IEEE Network 32(1), 64–69 (2018)

[17]. Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.: Survey on mobile edge computing. IEEE Communications Surveys & Tutorials 19(4), 2322–2358 (2017)

[18]. Carroll, A., Heiser, G.: Analysis of power consumption in smartphones. In: USENIX ATC 2010, pp. 21–21. USENIX (2010)

[19]. Pathak, A., Hu, Y.C., Zhang, M.: Where is the energy spent? In: EuroSys 2012, pp. 29–42. ACM (2012)